

SSAFY

Database

Many To Many Relationships 02

Python



목 차

팔로우 기능 구현

- 프로필 페이지
- 모델 관계 설정
- 기능 구현

Fixtures

- dumpdata
- loaddata

목 차

Improve query

- 사전 준비
- annotate
- select_related
- prefetch_related
- select_related & prefetch_related

참고

- 'exists' method
- 한꺼번에 dump 하기
- loaddata 인코딩 에러

학습 시작

Django에서 유저 관계를 정의하고 쿼리 성능을 높이는 방법을 고민해보자.

- 팔로우 기능을 구현하려면 유저 간 N:M 관계 설계가 필요합니다.
- 프로필 페이지를 통해 유저의 활동 정보(게시글, 댓글)를 효율적으로 조회해야 합니다.

1. 유저 간 팔로우 기능을 구현하려면 자기 자신을 참조하는 관계(self 관계)를 이해해야 합니다.
2. 다대다 관계 구성 시 역참조명 충돌을 방지하기 위해 [redacted] 을 설정합니다.
3. [redacted] 를 활용하면 팔로우 테스트 데이터를 손쉽게 재사용할 수 있습니다.
4. 많은 유저 데이터를 다룰 때 N+1 문제를 [redacted], [redacted], [redacted] 를 이용하여 해결할 수 있습니다.

- 복잡한 관계에서도 exists 메서드로 존재 여부를 빠르게 체크할 수 있습니다.
- 관계 설계와 쿼리 성능 향상을 통해 Django 웹앱의 품질을 크게 높일 수 있습니다.

학습 목표

- ✓ 유저 간 팔로우 기능을 위한 N:M 모델 관계를 설계하고 구현할 수 있다.
- ✓ fixtures(dumpdata, loaddata)를 활용하여 데이터 입출력을 효율화할 수 있다.
- ✓ annotate, select_related, prefetch_related를 사용해 N+1 문제를 해결할 수 있다.
- ✓ exists 등의 유틸 메서드를 활용한 성능 개선 방안을 이해한다.
- ✓ 복잡한 쿼리 요구 상황에서 Django ORM 최적화 전략을 적용할 수 있다

팔로우 기능 구현

프로필 페이지

프로필 페이지

- 각 회원의 개인 프로필 페이지에 팔로우 기능을 구현하기 위해 프로필 페이지를 먼저 구현하기
- 프로필 페이지에 해당 사용자의 추가 정보를 같이 볼 수 있게 내용 추가
 - 해당 사용자가 작성한 게시글 목록
 - 해당 사용자가 작성한 댓글 목록
 - 해당 사용자가 좋아요를 누른 게시글 목록

프로필 페이지 구현 (1/6)

- 프로필 페이지로 접근하기 위한 URL 주소 작성
 - 프로필 페이지는 사용자와 관련된 기능이기 때문에 'accounts' 앱의 'urls.py'에 추가
 - 로그인한 유저 뿐만 아니라 다른 사람의 프로필 페이지 방문도 가능해야 함
 - 사용자의 username을 variable routing을 활용하여 정보 전달이 필요

```
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    path('login/', views.login, name='login'),
    path('logout/', views.logout, name='logout'),
    ... (중략)

    path('profile/<username>/', views.profile, name='profile'),
]
```

프로필 페이지 구현 (2/6)

- view 함수 작성
 - URL로 전달되는 username 을 이용하여 사용자 정보를 조회하고 이를 template으로 전달
 - 유저 모델은 직접 import 하지 않고, get_user_model() 함수를 사용해야 함

```
# accounts/views.py

from django.contrib.auth import get_user_model

def profile(request, username):
    User = get_user_model()
    person = User.objects.get(username=username)
    context = {
        'person': person,
    }
    return render(request, 'accounts/profile.html', context)
```

프로필 페이지 구현 (3/6)

- 프로필 페이지의 세부 내용을 작성

```
<!-- accounts/profile.html -->
```

```
<h1>{{ person.username }}님의 프로필</h1>
<hr>
```

```
<h2>{{ person.username }}가 작성한 게시글</h2>
{% for article in person.article_set.all %}
    <div>{{ article.title }}</div>
{% endfor %}
<hr>
```

```
<!-- 이어서 -->
```

```
<h2>{{ person.username }}가 작성한 댓글</h2>
{% for comment in person.comment_set.all %}
    <div>{{ comment.content }}</div>
{% endfor %}
<hr>
```

```
<h2>{{ person.username }}가 좋아요한 게시글</h2>
{% for article in person.like_articles.all %}
    <div>{{ article.title }}</div>
{% endfor %}
```


프로필 페이지 구현 (4/6)

- index 페이지에서 내 프로필로 이동할 수 있도록 프로필 링크 추가

```
<!-- articles/index.html -->
<h1>Articles</h1>
{% if request.user.is_authenticated %}
  <h3>Hello, {{ user.username }}</h3>
  <a href="{% url 'accounts:profile' user.username %}">내 프로필</a>
  <a href="{% url 'articles:create' %}">NEW</a>

  ... (중략)
{% endif %}
<hr>
```

Articles

Hello, admin

[내 프로필](#) [NEW](#)

Logout

회원탈퇴

[회원정보 수정](#)

<그림1_DB_Many To Many Relationships 02_프로필 링크 확인 1>

프로필 페이지 구현 (5/6)

- 다른 유저의 프로필 페이지로 이동할 수 있도록 유저 아이디에도 해당 유저의 프로필 링크 추가

```
<!-- articles/index.html -->
```

```
... (중략)
```

```
{% for article in articles %}
```

```
<p>작성자 : <a href="{% url 'accounts:profile' article.user.username %}">  
    {{ article.user }}</a></p>
```

```
<p>글 번호: {{ article.pk }}</p>
```

```
... (중략)
```

```
<hr>
```

```
{% endfor %}
```

작성자 : [admin](#)

글 번호: 1

글 제목: [First](#)

글 내용: First Content

좋아요

<그림2_DB_Many To Many Relationships 02_프로필 링크 확인 2>

프로필 페이지 구현 (6/6)

- 프로필 페이지로 이동해서 데이터가 잘 나오는지 확인



모델 관계 설정

팔로우 기능의 모델 관계

User(M) - User(N)

팔로우는 유저와 유저와의 관계를 나타냄

- 회원은 여러 명의 회원을 팔로우 할 수 있고, 한 명도 하지 않을 수도 있음
- 회원은 여러 명의 팔로워를 가질 수 있고, 한 명도 가지지 않을 수도 있음

팔로우 모델 관계 설정 (1/2)

- 커스텀한 User 모델 클래스에 ManyToManyField를 사용하여 팔로우 필드를 추가
 - User 모델과 관계를 맺는 것이기 때문에 settings.AUTH_USER_MODEL을 사용해도 되지만 자기 자신과의 관계이기 때문에 'self'로 표현할 수 있음
 - 팔로우 기능은 단방향의 관계이기 때문에 반드시 symmetrical 속성을 False로 설정
 - 참조 필드는 followings 필드로 내가 팔로우하는 사람들을 의미
 - 역참조 필드는 user_set을 사용해도 되지만 명확한 설정을 위해 related_name을 이용하여 'followers'로 변경

```
# accounts/models.py
```

```
class User(AbstractUser):  
    followings = models.ManyToManyField('self', symmetrical=False, related_name='followers')
```

※ 참조, 역참조는 서로 바뀌어도 상관없으나 관계 조회시 생각하기 편한 방향으로 정하여 작성됨

팔로우 모델 관계 설정 (2/2)

- Migration 진행 후 중개 테이블이 생성되었는지 확인

```

└─ accounts_user_followings
  └─ Columns
    ├── id INTEGER
    ├── from_user_id bigint
    └─ to_user_id bigint

```

<그림4_DB_Many To Many Relationships 02_팔로우 테이블 구조 확인>

기능 구현

팔로우 기능 구현 (1/5)

- 팔로우 기능을 위한 URL 추가
 - 어느 사용자를 팔로우 하는지 상대 사용자에게 대한 정보를 variable routing을 활용하여 전달

```
# accounts/urls.py

app_name = 'accounts'
urlpatterns = [
    path('login/', views.login, name='login'),

    ... (중략)

    path('profile/<username>/', views.profile, name='profile'),
    path('<int:user_pk>/follow/', views.follow, name='follow'),
]
```

팔로우 기능 구현 (2/5)

- 중개 테이블에 이미 내 정보가 있으면 제거 (팔로우 해제)
- 중개 테이블에 내 정보가 없으면 추가 (팔로우 진행)

```
# accounts/views.py
@login_required
def follow(request, user_pk):
    User = get_user_model()
    person = User.objects.get(pk=user_pk)

    if person != request.user:
        if request.user in person.followers.all():
            person.followers.remove(request.user)
        else:
            person.followers.add(request.user)

    return redirect('accounts:profile', person.username)
```


팔로우 기능 구현 (3/5)

- 프로필 페이지에서 팔로우 기능을 추가
 - 팔로우와 팔로잉이 몇 명 있는지 DTL 을 사용하여 출력하는 부분 추가

```
<!-- accounts/profile.html -->
```

```
<h1>{{ person.username }}님의 프로필</h1>
```

```
<div>
```

```
<div>
```

```
    팔로잉 : {{ person.followings.all|length }} / 팔로워 : {{ person.followers.all|length }}
```

```
</div>
```

```
... (중략)
```

팔로우 기능 구현 (4/5)

- 프로필 페이지에서 팔로우 기능을 추가
 - 프로필 페이지의 유저(person)가 로그인한 사용자(request.user)의 페이지인 경우 팔로우 버튼 숨기기
 - 팔로우를 수행할 버튼을 추가하고 팔로우 여부에 따라 버튼의 텍스트를 다르게 출력

<!-- 이어서 작성 -->

```
{% if request.user != person %}
<div>
  <form action="{% url 'accounts:follow' person.pk %}" method="POST">
    {% csrf_token %}
    {% if request.user in person.followers.all %}
      <input type="submit" value="Unfollow">
    {% else %}
      <input type="submit" value="Follow">
    {% endif %}
  </form>
</div>
{% endif %}
```

팔로우 기능 구현 (5/5)

• 팔로우 기능 확인

- 팔로우 버튼을 눌렀을 때 숫자와 버튼 텍스트
- 내 프로필에서 팔로우 버튼 생략



<그림5_DB_Many To Many Relationships 02_팔로우 기능 확인>

• 팔로우 버튼 클릭 후 중개 테이블 데이터 확인

- from_user_id 는 버튼을 누른 유저 (나)
- to_user_id 는 팔로우된 유저 (상대방)

id	from_user_id	to_user_id
INTEGER	bigint	bigint
1	1	2

<그림6_DB_Many To Many Relationships 02_팔로우 데이터 확인>

Fixtures

Fixtures

Django 개발 시 데이터 베이스 초기화 및 공유를 위해
사용되는 파일 형식

Fixtures 사용 목적

- 초기 데이터 세팅
 - 웹 서비스가 처음 시작될 때 필요한 기본 데이터(기본 권한 그룹, 상품 카테고리 등)를 미리 세팅할 수 있음
- 테스트 샘플 데이터 준비
 - 테스트할 때, 항상 동일하고 예측 가능한 데이터 환경을 구축하여 테스트의 신뢰성과 반복 가능성을 높이는 데 활용
- 협업 시 동일한 데이터 환경 맞추기
 - 팀원들이 각자의 개발 환경을 설정할 때, 모두 동일한 초기 데이터나 특정 테스트 데이터 셋을 쉽게 공유하고 적용
 - 개발 환경 간의 일관성을 유지하고 협업 효율을 높이는 데 도움을 줌

초기 데이터의 필요성

- 협업하는 유저 A, B가 있다고 생각 했을 때
 1. A가 먼저 프로젝트를 작업 후 원격 저장소에 push 진행
 - .gitignore로 인해 DB는 업로드하지 않기 때문에 A가 생성한 데이터도 업로드 X
 2. B가 원격 저장소에서 A가 push한 프로젝트를 pull (혹은 clone)
 - 결과적으로 B는 DB가 없는 프로젝트를 받게 됨
- 이처럼 프로젝트의 앱을 처음 설정할 때 동일하게 준비 된 데이터로 데이터베이스를 미리 채우는 것이 필요한 순간이 있음
- Django에서는 fixtures를 사용해 앱에 초기 데이터(initial data)를 제공

fixtures 관련 명령어

- 'dumpdata'
 - 데이터베이스에서 데이터를 내보낼 때 사용하는 명령어
 - 주로 파일(JSON) 형태로 추출함
 - 특정 테이블의 데이터만 추출도 가능함
- 'loaddata'
 - 데이터베이스에 데이터를 불러올 때 사용하는 명령어
 - 내보내기 형태로 저장된 JSON을 읽어와서 데이터베이스에 저장함

Fixtures 실습 사전 준비

- M:N 까지 모두 작성된 Django 프로젝트에서 유저, 게시글, 댓글 데이터 준비하기
 - 유저 데이터
 - 최소 3명의 유저를 회원 가입을 통해 준비
 - 게시글
 - 유저 별로 1~2개의 게시글 생성을 통해 데이터 준비
 - 댓글
 - 게시글 당 2~3개씩 작성하여 데이터 준비
 - 유저 별로 0~2개 정도 작성되도록 준비

dumpdata

dumpdata

데이터베이스의 특정 모델 혹은 앱 전체 데이터를 추출

dumpdata 기본 명령어

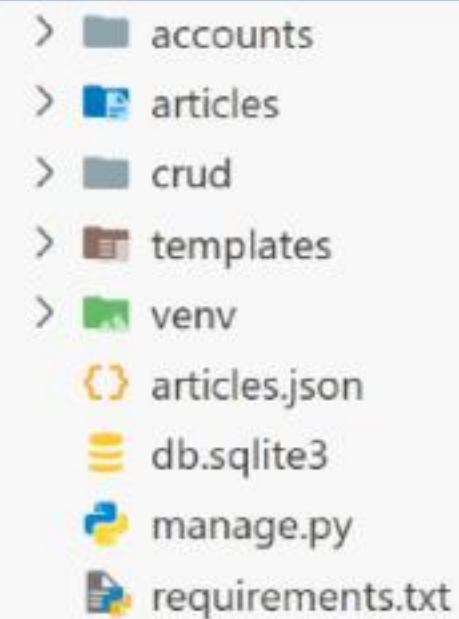
```
$ python manage.py dumpdata [앱이름.모델이름] [옵션] > 추출파일명.json
```

- 앱이름만 지정
 - 해당 앱의 모든 모델에 대한 데이터를 추출
- 앱이름.모델이름 지정
 - 특정 모델의 데이터를 추출
- 앱 혹은 모델명을 지정하지 않은 경우
 - 프로젝트 전체의 모델 데이터를 추출
- --format 옵션을 통해 JSON, YAML 등의 형식 지정 가능(기본값: JSON)

dumpdata 명령어 예시 (1/2)

```
$ python manage.py dumpdata --indent 4 articles.article > articles.json
```

- articles 앱의 Article 모델 데이터를 추출
 - '--indent 4' 는 들여쓰기를 4칸을 한다는 옵션
- 명령어 실행 후 프로젝트 폴더에 articles.json 파일이 생성됨
- articles.json 파일에는 Article 모델의 모든 데이터가 JSON 형식으로 작성되어 있음
- Fixtures 파일명은 자유롭게 작성 가능



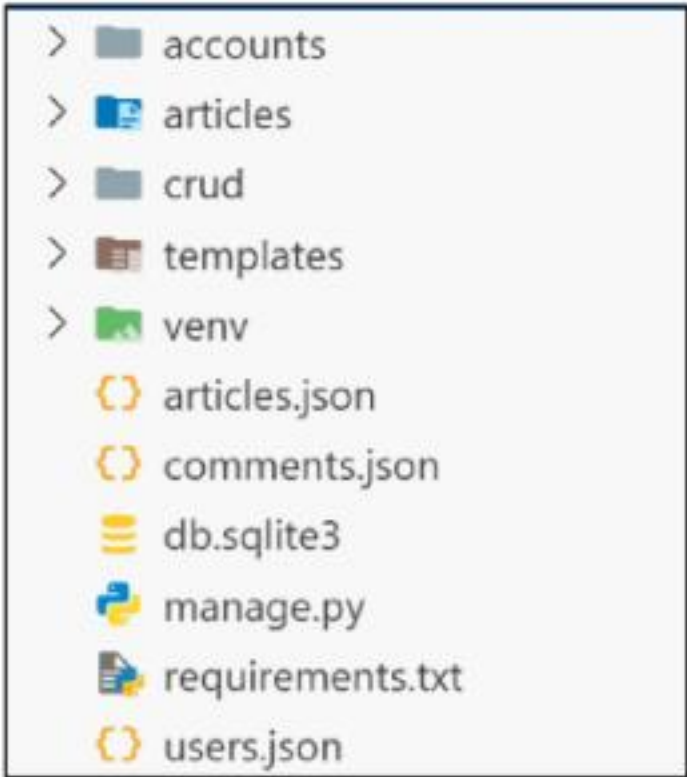
```
> accounts
> articles
> crud
> templates
> venv
> articles.json
> db.sqlite3
> manage.py
> requirements.txt
```

<그림7_DB_Many To Many Relationships 02_생성된 articles.json 파일>

dumpdata 명령어 예시 (2/2)

- article앱의 comment 모델과 account 앱의 user 모델의 데이터를 추출

```
$ python manage.py dumpdata --indent 4 articles.comment > comments.json  
$ python manage.py dumpdata --indent 4 accounts.user > users.json
```



- > accounts
- > articles
- > crud
- > templates
- > venv
- articles.json
- comments.json
- db.sqlite3
- manage.py
- requirements.txt
- users.json

<그림8_DB_Many To Many Relationships 02_생성된 fixture 파일들>

dumpdata 정리

- dumpdata 명령어를 사용하면 프로젝트 내 특정 앱 혹은 모델에 대한 데이터를 JSON 등 원하는 포맷으로 추출 가능
- 이렇게 생성된 데이터 파일은 추후 다른 환경에서 loaddata로 불러와 동일한 데이터 상태를 재현할 수 있으며, 협업 및 배포에 큰 장점이 있음

loaddata

 loaddata

loaddata

dumpdata를 통해 추출한 데이터 파일을
다시 데이터베이스에 반영

loaddata 기본 명령어

- Fixtures 파일의 기본 경로에 있는 파일을 DB에 반영

```
$ python manage.py loaddata 파일경로
```

- Fixtures 파일의 기본 경로
 - `app_name/fixtures/`
- Django는 설치된 모든 app의 디렉토리에서 fixtures 폴더 이후의 경로로 fixtures 파일을 찾아 load를 진행

loaddata 실습 사전 준비

- 'articles' 폴더 내에 'fixtures' 폴더를 생성후 해당 폴더에 dumpdata로 생성한 파일들을 위치



<그림9_DB_Many To Many Relationships 02_fixtures 폴더 내 fixture 파일>

- 'db.sqlite3' 파일 삭제 후 migrate 진행하여 데이터베이스 초기화

loaddata 명령어 예시 (1/2)

- dumpdata로 생성한 파일들을 모두 DB에 반영
- 파일을 한 번에 작성하면, Django가 자동으로 우선 순위가 높은 파일부터 불러오므로 json 파일의 작성 순서는 상관없음

```
$ python manage.py loaddata articles.json users.json comments.json
```


loaddata 명령어 예시 (2/2)

- 단, loaddata를 한번에 실행하지 않고 별도로 실행한다면 모델 관계에 따라 load 순서가 중요할 수 있음
 - comment는 article에 대한 key 및 user에 대한 key가 필요
 - article은 user에 대한 key가 필요
- 즉, 현재 모델 관계에서는 user → article → comment 순으로 data를 load 해야 오류가 발생하지 않음

```
$ python manage.py loaddata users.json  
$ python manage.py loaddata articles.json  
$ python manage.py loaddata comments.json
```

- 첫 번째로 comments.json를 먼저 loaddata 하게 되면 아래와 같이 에러가 발생
 - 게시글 정보를 찾을 수 없어 에러가 발생

```
django.db.utils.IntegrityError: Problem installing fixtures: The row in table  
'articles_comment' with primary key '1' has an invalid foreign key:  
articles_comment.user_id contains a value '1' that does not have a corresponding value in  
accounts_user.id.
```

loaddata 주의 사항

- loaddata를 실행하기 전에 해당 모델에 대한 마이그레이션이 완료되어 있어야 함
- 같은 PK를 가진 데이터가 이미 있는 경우 중복 에러가 발생할 수 있음
 - 이 경우 기존 데이터를 지우거나, 새로운 Fixture 파일을 사용해야 함

loaddata 정리

- loaddata 명령어는 dumpdata로 추출한 Fixture 파일을 DB로 불러오는 명령어이며, 개발 환경 준비나 협업 시 매우 유용
- 마이그레이션 상태를 먼저 확인하고, 인코딩 문제 등을 사전에 해결하면 매끄럽게 데이터를 복원할 수 있음

Improve query

Improve query

Improve query

같은 결과를 얻기 위해 DB 측에 보내는
query 개수를 점차 줄여 조회하기
➤ N+1 Problem을 해결할 수 있음

N+1 Problem

N+1 Problem

1개의 쿼리로 데이터를 가져왔더라도 관련 데이터를 추가로 가져오기 위해
추가 쿼리가 N개 더 실행되는 상황

N+1 문제는 ORM에서 관련 객체를 반복적으로 접근할 때 과도한 쿼리가 발생하는 현상입니다.

초기 1번의 쿼리로 기본 데이터를 조회한 뒤, 각 객체의 연결된 데이터를 조회하며 N번의 추가 쿼리가 실행됩니다.

이로 인해 총 N+1개의 쿼리가 발생하고, 데이터가 많을수록 성능 저하나 DB 부하가 심해질 수 있습니다.

※ 이어지는 실습에서 N+1 Problem을 확인할 수 있음

사전 준비

Improve query 실습 사전 준비 (1/2)

- 99-improve-query 프로젝트에서 진행
- fixtures 데이터
 - 게시글 10개 / 댓글 100개 / 유저 5개
- 모델 관계
 - N:1 - Article:User / Comment:Article / Comment:Article
 - N:M - Article:User

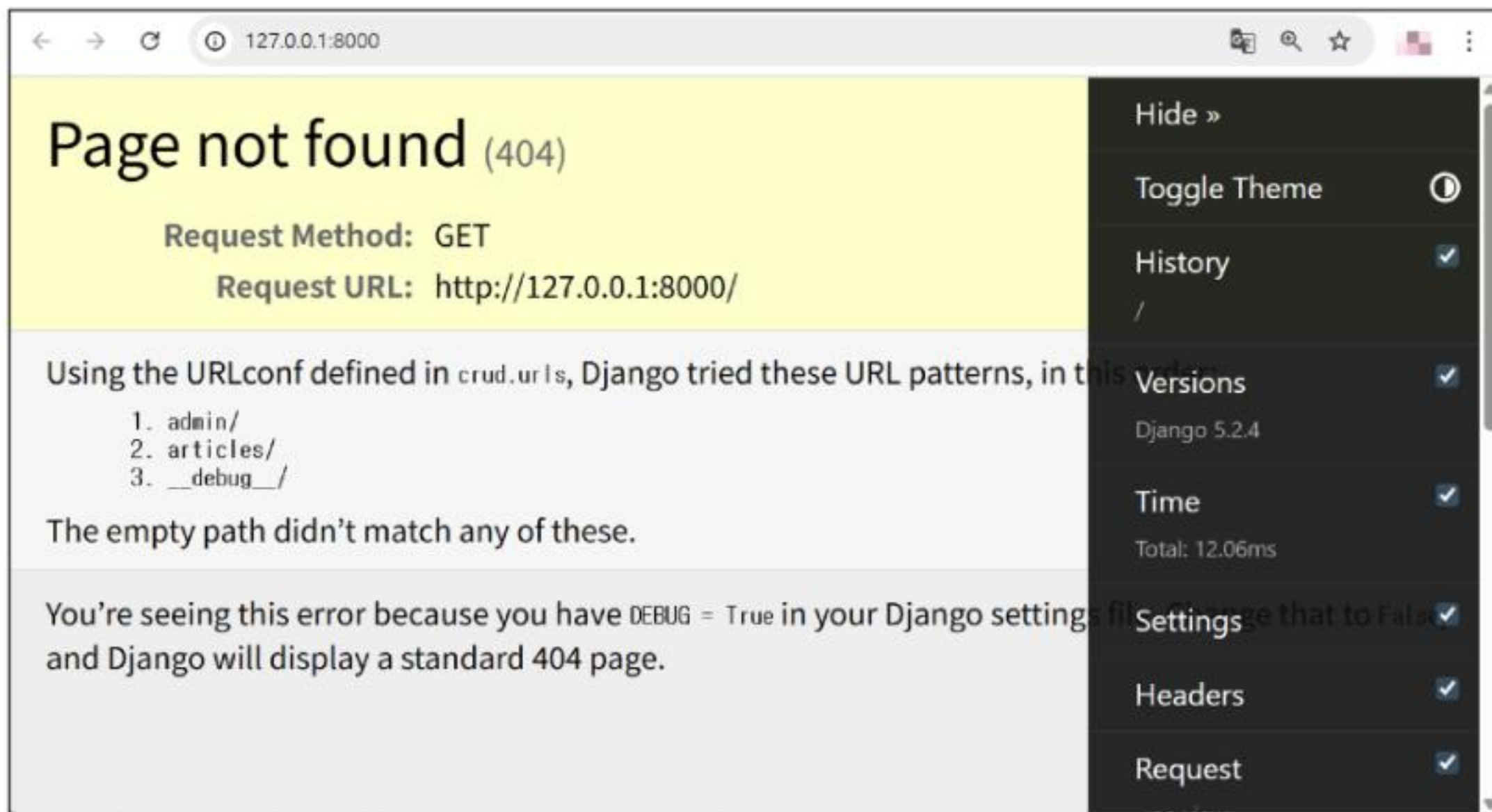
```
$ python manage.py migrate
```

```
$ python manage.py loaddata users.json articles.json comments.json
```

```
Installed 115 object(s) from 3 fixture(s)
```

Improve query 실습 사전 준비 (2/2)

- Django Debug Toolbar 실행 확인



annotate

Django annotate

- SQL의 GROUP BY를 사용
 - 각 행(row)별로 계산된 필드를 추가함
- 쿼리셋의 각 객체에 계산된 필드를 추가
 - 기존 필드에 새로운 필드를 추가하여 계산된 값을 넣어 반환
- 집계 함수(Count, Sum, Avg, Max, Min 등)와 함께 자주 사용됨



집계 함수
여러 데이터를 요약하여 분석할
때 사용하는 함수

<개념1_DB_Many To Many Relationships 02_ 집계 함수>

annotate 예시

```
Book.objects.annotate(num_authors=Count('authors'))
```

• 의미

- 결과 객체에 'num_authors'라는 새로운 필드를 추가
- 이 필드는 각 책과 연관된 저자의 수를 계산

• 결과

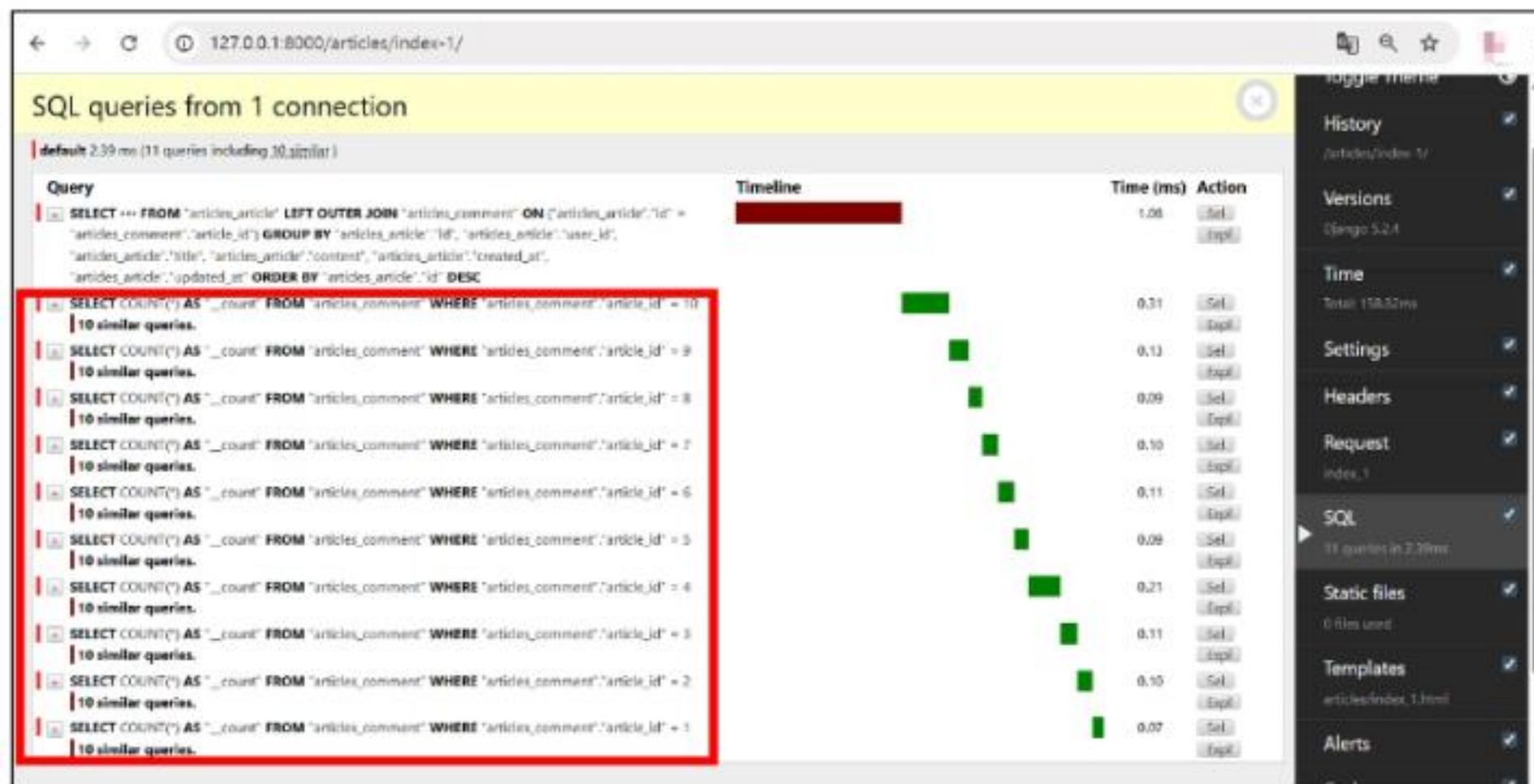
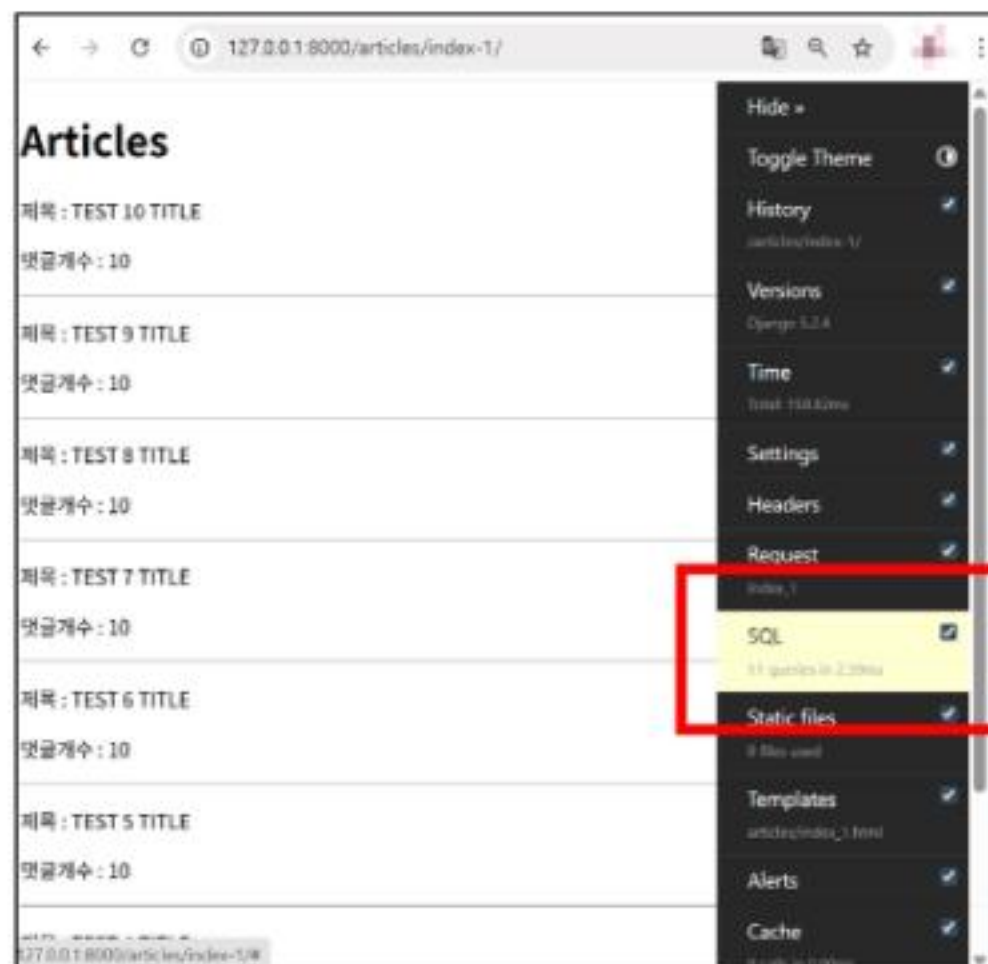
- 결과에는 기존 필드와 함께 'num_authors' 필드를 가지게 됨
- book.num_authors로 해당 책의 저자 수에 접근할 수 있게 됨

TIP

- 'num_authors' 속성은 새롭게 추가되는 필드 명을 의미하며 다른 명칭으로 작성해도 됩니다.
- 만약 속성을 작성하지 않고 생략하면 'authors__count' 로 필드명을 알아서 생성하게 됩니다.
- 새롭게 생성되는 필드 'num_authors' 는 book.num_authors로 해당 책의 저자 수에 접근할 수 있습니다.

N+1 Problem 상황 (1/2)

- <http://127.0.0.1:8000/articles/index-1/>
 - 11 queries including 10 similar



〈그림11_DB_Many To Many Relationships 02_댓글 개수 조회 쿼리〉

N+1 Problem 상황 (2/2)

- 문제의 원인

- 전체 게시글 쿼리 1개와 각 게시글 댓글 개수 확인을 위해 10개의 쿼리가 추가로 요청
- 각 게시글의 댓글 개수를 계산할 때, 매 번 데이터베이스로 쿼리를 보내 댓글 데이터를 받아오고 있음
- 만약 게시글이 100개이고 요청을 10명, 100명이 동시에 하면 그 만큼 서버에 부하가 커짐

```
<!-- index_1.html -->
```

```
<p>댓글개수 : {{ article.comment_set.count }}</p>
```

annotate 적용 (1/2)

- 문제 해결

- 게시글을 조회하면서 댓글 개수까지 한번에 조회해서 가져오기
 - annotate의 집계 함수를 이용해서 새로운 필드에 댓글의 개수를 추가해서 반환

```
# articles/views.py

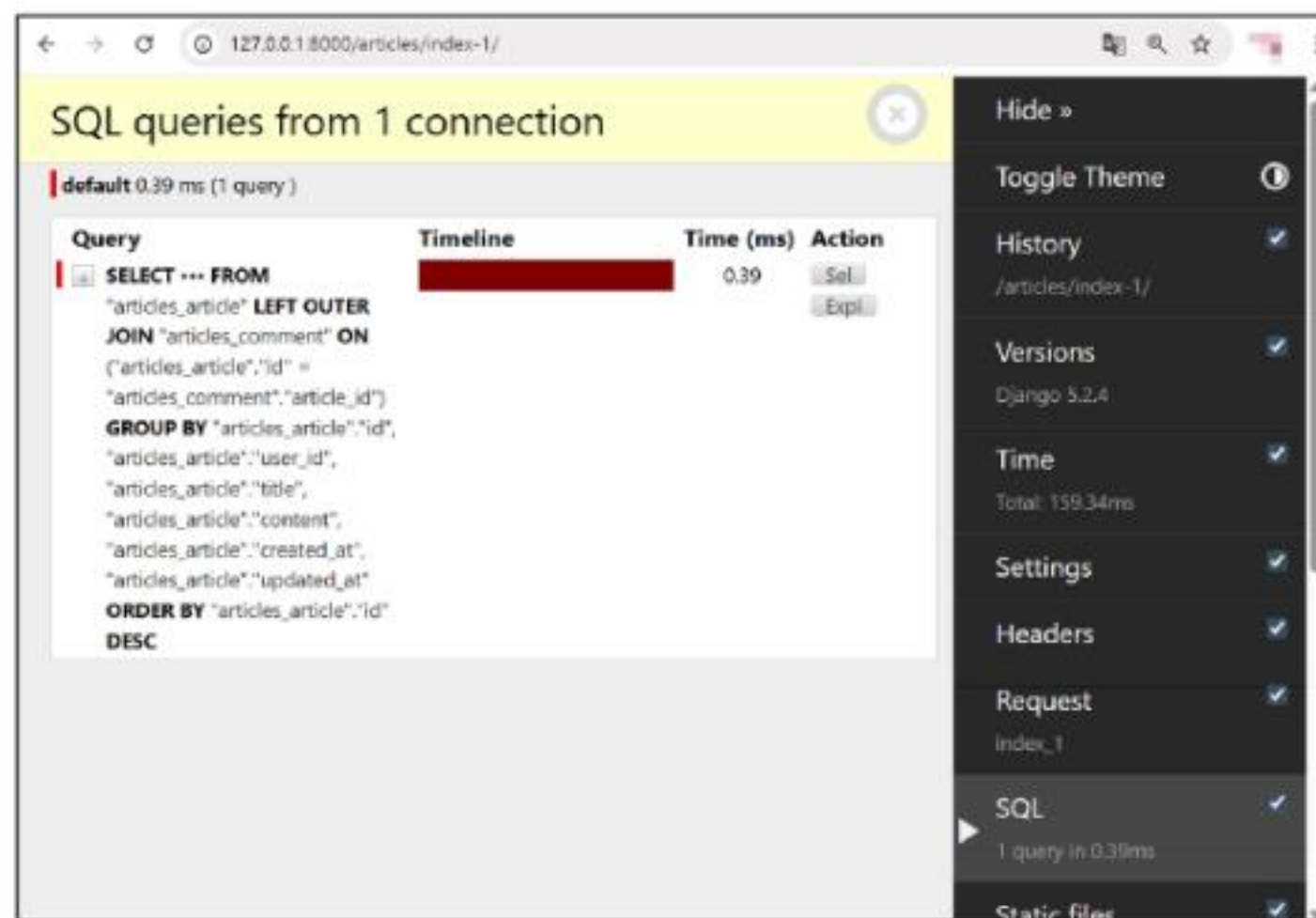
def index_1(request):
    # articles = Article.objects.order_by('-pk')
    articles = Article.objects.annotate(comment_count=Count('comment')).order_by('-pk')
    context = {
        'articles': articles,
    }
    return render(request, 'articles/index_1.html', context)
```

```
<!-- index_1.html -->
```

```
<p>댓글개수 : {{ article.comment_count }}</p>
```


annotate 적용 (2/2)

- 문제 해결
 - "11 queries including 10 similar" => "1 query"



<그림12_DB_Many To Many Relationships 02_댓글 조회 중복 쿼리 해결>

select_related

select_related

select_related

- ForeignKey 또는 OneToOne 관계에서 사용되는 Django ORM의 메서드
- ForeignKey, OneToOne 관계에서 **내부적으로 INNER JOIN을 사용하여 하여 관련 객체를 한 번에 불러옴**
- 단일 쿼리로 관련 객체를 함께 가져와 성능을 향상

select_related 예시

```
Book.objects.select_related('publisher')
```

• 의미

- Book 모델과 연관된 Publisher 모델의 데이터를 함께 가져옴
- ForeignKey 관계인 'publisher'를 JOIN하여 단일 쿼리만으로 데이터를 조회

• 결과

- Book 객체를 조회할 때 연관된 Publisher 정보도 함께 로드
- book.publisher.name과 같은 접근을 **추가적인 데이터베이스 쿼리 없이** 가능

N+1 Problem 상황 (1/2)

- <http://127.0.0.1:8000/articles/index-2/>
 - "11 queries including 10 similar and 8 duplicates"

Articles

작성자 : murphysusan

제목 : TEST 10 TITLE

작성자 : marshallpatricia

제목 : TEST 9 TITLE

작성자 : marshallpatricia

제목 : TEST 8 TITLE

작성자 : murphysusan

제목 : TEST 7 TITLE

Hide »

Toggle Theme

History

Versions

Time

Settings

Headers

Request

SQL

Static files

SQL queries from 1 connection

default 1.42 ms (11 queries including 10 similar and 8 duplicates)

Query	Timeline	Time (ms)	Action
SELECT ... FROM "articles_article" ORDER BY "articles_article"."id" DESC		0.35	Set Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 5 LIMIT 21		0.20	Set Expl
10 similar queries. Duplicated 3 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21		0.15	Set Expl
10 similar queries. Duplicated 3 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21		0.11	Set Expl
10 similar queries. Duplicated 3 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 5 LIMIT 21		0.10	Set Expl
10 similar queries. Duplicated 3 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 2 LIMIT 21		0.08	Set Expl
10 similar queries. Duplicated 2 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21		0.09	Set Expl
10 similar queries. Duplicated 3 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 2 LIMIT 21		0.09	Set Expl
10 similar queries. Duplicated 2 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 4 LIMIT 21		0.10	Set Expl
10 similar queries.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 5 LIMIT 21		0.08	Set Expl
10 similar queries. Duplicated 3 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 3 LIMIT 21		0.09	Set Expl
10 similar queries.			

N+1 Problem 상황 (2/2)

• 문제 원인

- 게시글의 작성자를 확인하기 위해 추가로 데이터베이스에 요청되고 있음
- 같은 작성자가 작성한 여러 개의 게시글들도 각각 작성자 정보를 조회하기 위해 동일하게 쿼리를 요청하여 중복 쿼리가 발생하고 있음
- 해당 요청도 많은 사용자가 요청하는 경우 중복된 쿼리로 인해 서버의 성능이 저하됨

```
<!-- index_2.html -->
```

```
{% for article in articles %}  
  <h3>작성자 : {{ article.user.username }}</h3>  
  <p>제목 : {{ article.title }}</p>  
  <hr>  
{% endfor %}
```

select_related 적용 (1/2)

- 문제 해결
 - 게시글을 조회하면서 **유저 정보까지 한번에 조회**해서 가져오기

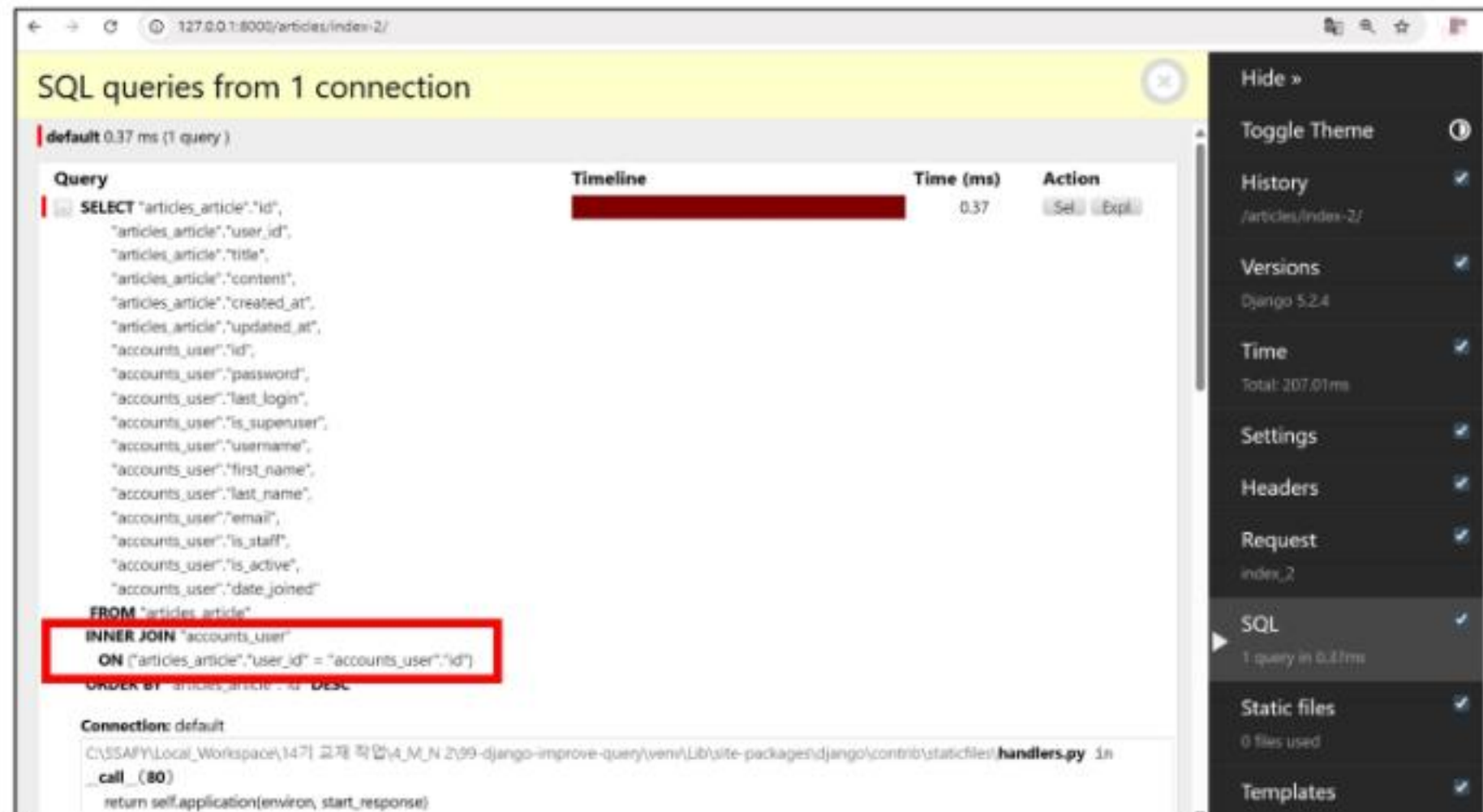
```
# articles/views.py

def index_2(request):
    # articles = Article.objects.order_by('-pk')
    articles = Article.objects.select_related('user').order_by('-pk')
    context = {
        'articles': articles,
    }
    return render(request, 'articles/index_2.html', context)
```


select_related 적용 (2/2)

• 문제 해결

- "11 queries including 10 similar and 8 duplicates" => "1 query"



<그림14_DB_Many To Many Relationships 02_게시글의 작성자 조회 중복 쿼리 해결>

prefetch_related

 prefetch_related

prefetch_related

- SQL이 아닌 Python을 사용한 JOIN을 진행
 - 관련 객체들을 미리 가져와 메모리에 저장하여 성능을 향상
- M:N 또는 N:1 역참조 관계에서 사용
 - ManyToManyField나 역참조 관계에 대해 별도의 쿼리를 실행

prefetch_related 예시

```
Book.objects.prefetch_related('authors')
```

• 의미

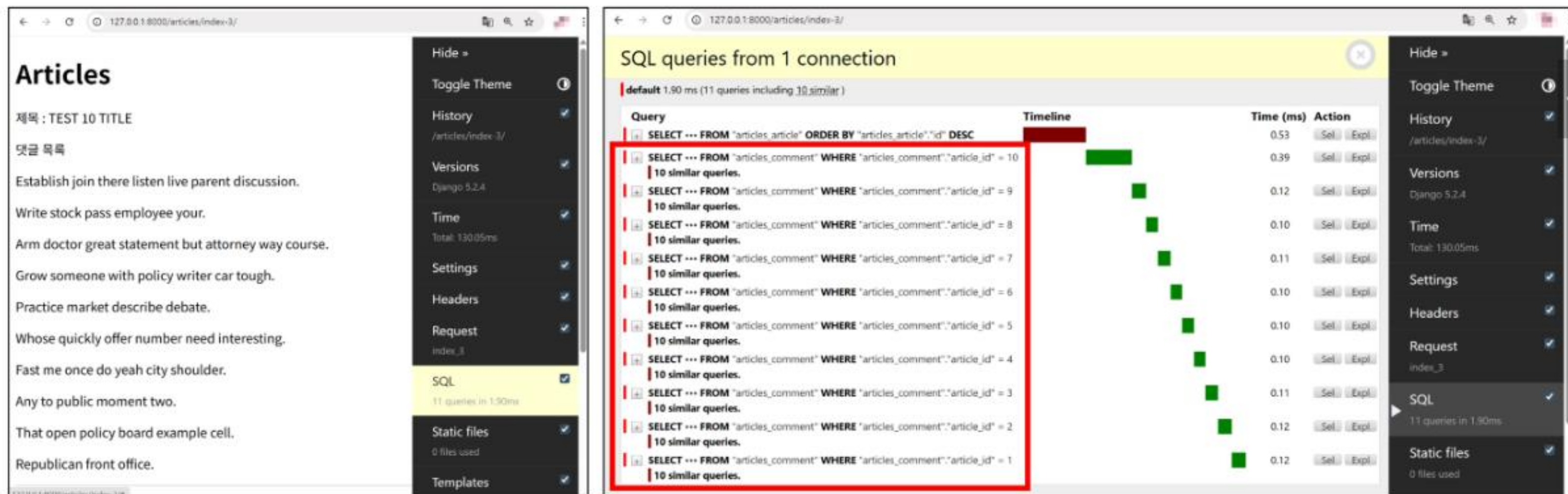
- Book과 Author는 ManyToMany 관계로 가정
- Book 모델과 연관된 모든 Author 모델의 데이터를 미리 가져옴
- Django가 별도의 쿼리로 Author 데이터를 가져와 관계를 설정

• 결과

- Book 객체들을 조회한 후, 연관된 모든 Author 정보가 미리 로드 됨
- `for author in book.authors.all()`와 같은 반복이 추가적인 데이터베이스 쿼리 없이 실행됨

N+1 Problem 상황 (1/2)

- <http://127.0.0.1:8000/articles/index-3/>
- "11 queries including 10 similar"



N+1 Problem 상황 (2/2)

• 문제 원인

- 게시글마다 댓글이 존재하므로, 댓글 정보를 확인하기 위해 추가적인 데이터베이스 요청이 발생
- 이러한 요청은 게시글 수에 따라 쿼리 수가 증가하게 됨
- 다수의 사용자가 동시에 요청할 경우, 서버의 성능 저하로 이어질 수 있음

```
<!-- index_3.html -->
{% for article in articles %}
  <p>제목 : {{ article.title }}</p>
  <p>댓글 목록</p>
  {% for comment in article.comment_set.all %}
    <p>{{ comment.content }}</p>
  {% endfor %}
<hr>
{% endfor %}
```

prefetch_related 적용 (1/2)

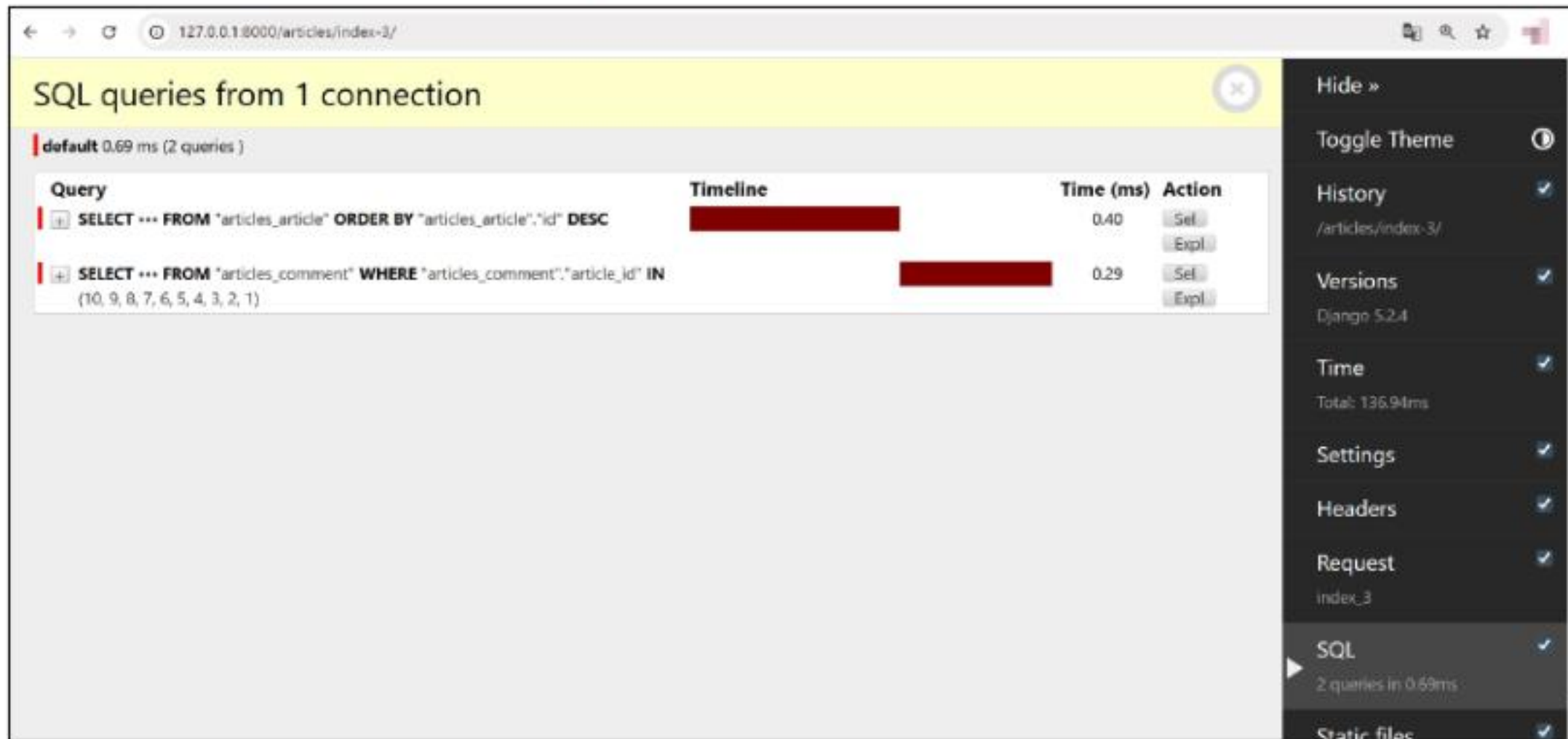
- 문제 해결
 - 게시글을 조회하면서 참조된 댓글까지 한번에 조회해서 가져오기

```
# articles/views.py

def index_3(request):
    # articles = Article.objects.order_by('-pk')
    articles = Article.objects.prefetch_related('comment_set').order_by('-pk')
    context = {
        'articles': articles,
    }
    return render(request, 'articles/index_3.html', context)
```

prefetch_related 적용 (2/2)

- 문제 해결
 - "11 queries including 10 similar" => "2 queries"



**select_related &
prefetch_related**

N+1 Problem 상황 (1/2)

- <http://127.0.0.1:8000/articles/index-4/>
- "111 queries including 110 similar and 100 duplicates"

Articles

제목 : TEST 10 TITLE

댓글 목록

marshallpatricia : Establish join there listen live parent discussion.

jimmybean : Write stock pass employee your.

marshallpatricia : Arm doctor great statement but attorney way course.

marshallpatricia : Grow someone with policy writer car tough.

margaret13 : Practice market describe debate.

marshallpatricia : Whose quickly offer number need interesting.

jimmybean : Fast me once do yeah city shoulder.

murphysusan : Any to public moment two.

margaret13 : That open policy board example cell.

marshallpatricia : Republican front office.

Hide »

Toggle Theme

History

Versions

Time

Settings

Headers

Request

SQL

Static files

Templates

SQL queries from 1 connection

default 11.67 ms (111 queries including 110 similar and 100 duplicates)

Query	Timeline	Time (ms)	Action
SELECT ... FROM "articles_article" ORDER BY "articles_article","id" DESC		0.46	Sel Expl
SELECT ... FROM "articles_comment" WHERE "articles_comment","article_id" = 10		0.16	Sel Expl
10 similar queries.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 1 LIMIT 21		0.21	Sel Expl
100 similar queries. Duplicated 27 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 4 LIMIT 21		0.14	Sel Expl
100 similar queries. Duplicated 17 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 1 LIMIT 21		0.13	Sel Expl
100 similar queries. Duplicated 27 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 1 LIMIT 21		0.11	Sel Expl
100 similar queries. Duplicated 27 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 2 LIMIT 21		0.09	Sel Expl
100 similar queries. Duplicated 15 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 1 LIMIT 21		0.09	Sel Expl
100 similar queries. Duplicated 27 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 4 LIMIT 21		0.09	Sel Expl
100 similar queries. Duplicated 17 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 5 LIMIT 21		0.09	Sel Expl
100 similar queries. Duplicated 20 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 2 LIMIT 21		0.07	Sel Expl
100 similar queries. Duplicated 15 times.			
SELECT ... FROM "accounts_user" WHERE "accounts_user","id" = 1 LIMIT 21		0.08	Sel Expl
100 similar queries. Duplicated 27 times.			

Hide »

Toggle Theme

History

Versions

Time

Settings

Headers

Request

SQL

Static files

Templates

N+1 Problem 상황 (2/2)

• 문제 원인

- "게시글" + "각 게시글의 댓글 목록" + "댓글의 작성자"를 단계적으로 평가
- 게시글 목록을 가져올 때 해당 게시글의 댓글 목록을 가져오기 위해 데이터베이스에 쿼리 요청
- 이 때 댓글의 작성자를 확인하기 위해 댓글마다 사용자 정보도 데이터베이스로 쿼리 요청

```
<!-- index_4.html -->
```

```
{% for article in articles %}
```

```
<p>제목 : {{ article.title }}</p>
```

```
<p>댓글 목록</p>
```

```
{% for comment in article.comment_set.all %}
```

```
<p>{{ comment.user.username }} : {{ comment.content }}</p>
```

```
{% endfor %}
```

```
<hr>
```

```
{% endfor %}
```


prefetch_related 적용 (1/2)

- 문제 해결 1단계
 - prefetch_related를 활용하여 게시글을 조회할 때 같이 참조된 댓글 까지 조회하도록 수정

```
# articles/views.py

def index_4(request):
    # articles = Article.objects.order_by('-pk')
    articles = Article.objects.prefetch_related('comment_set').order_by('-pk')

    context = {
        'articles': articles,
    }
    return render(request, 'articles/index_4.html', context)
```

prefetch_related 적용 (2/2)

- 문제 해결 1단계
 - "111 queries including 110 similar and 100 duplicates"
=> "102 queries including 100 similar and 100 duplicates"

SQL queries from 1 connection

default 13.28 ms (102 queries including 100 similar and 100 duplicates)

Query	Timeline	Time (ms)	Action
SELECT ... FROM "articles_article" ORDER BY "articles_article"."id" DESC		0.41	Sel Expl
SELECT ... FROM "articles_comment" WHERE "articles_comment"."article_id" IN (10, 9, 8, 7, 6, 5, 4, 3, 2, 1)		0.09	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21 100 similar queries. Duplicated 27 times.		0.36	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 4 LIMIT 21 100 similar queries. Duplicated 17 times.		0.18	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21 100 similar queries. Duplicated 27 times.		0.14	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21 100 similar queries. Duplicated 27 times.		0.13	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 2 LIMIT 21 100 similar queries. Duplicated 15 times.		0.12	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21 100 similar queries. Duplicated 27 times.		0.10	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 4 LIMIT 21 100 similar queries. Duplicated 17 times.		0.12	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 5 LIMIT 21 100 similar queries. Duplicated 20 times.		0.10	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 2 LIMIT 21 100 similar queries. Duplicated 15 times.		0.12	Sel Expl
SELECT ... FROM "accounts_user" WHERE "accounts_user"."id" = 1 LIMIT 21 100 similar queries. Duplicated 27 times.		0.10	Sel Expl

Hide +

Toggle Theme

History
/articles/index-4/

Versions
Django 5.2.4

Time
Total: 270.69ms

Settings

Headers

Request
index_4

SQL
102 queries in 13.28ms

Static files
0 files used

Templates



댓글을 조회할 때 각 댓글의 작성자 요청을
중복 조회하고 있음

select_related & prefetch_related 적용 (1/2)

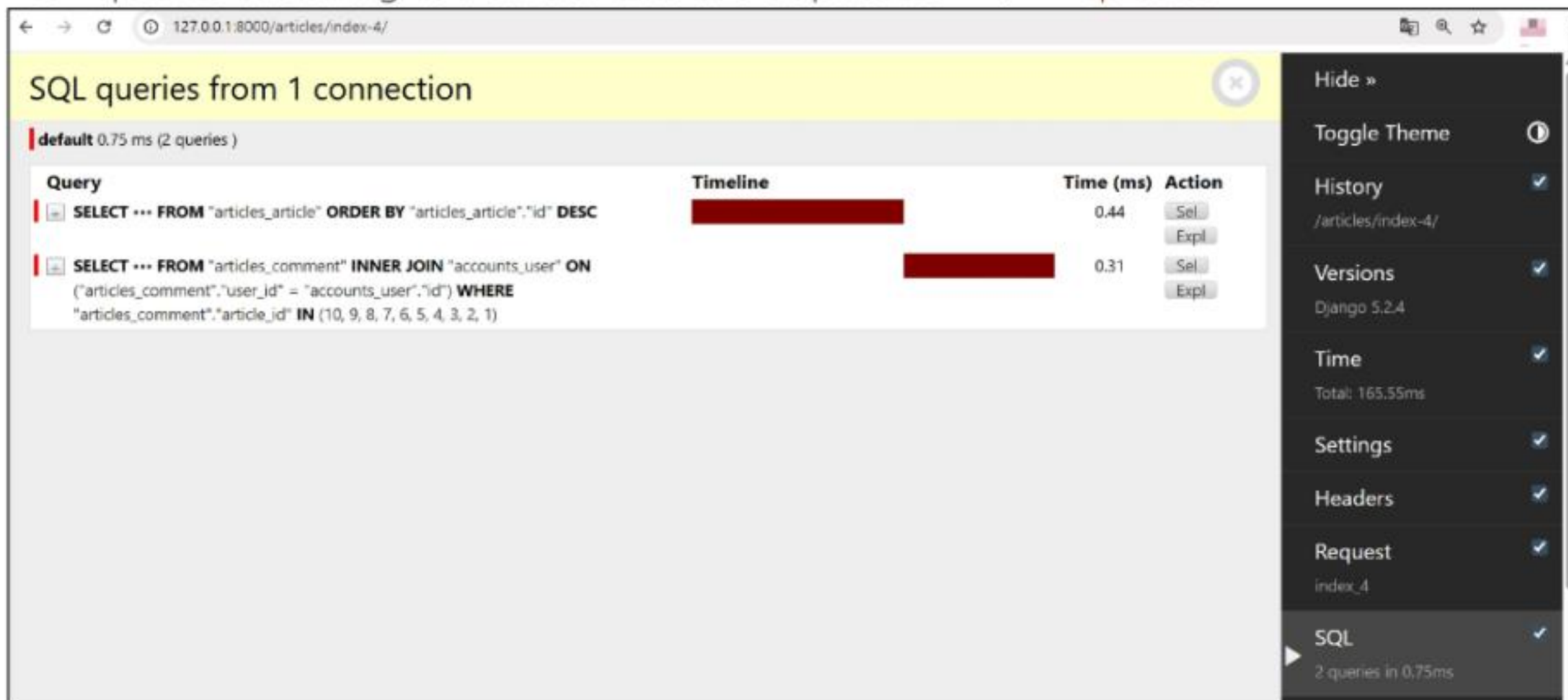
- 댓글 정보를 요청할 때 select_related를 활용하여 댓글 작성자 정보를 JOIN하여 가져온 다음 Prefetch 함수를 이용하여 게시글 정보와 같이 요청해서 가져오도록 수정

```
# articles/views.py
from django.db.models import Prefetch
def index_4(request):
    # articles = Article.objects.order_by('-pk')
    # articles = Article.objects.prefetch_related('comment_set').order_by('-pk')
    articles = Article.objects.prefetch_related(
        Prefetch('comment_set', queryset=Comment.objects.select_related('user'))
    ).order_by('-pk')
    context = {
        'articles': articles,
    }
    return render(request, 'articles/index_4.html', context)
```

select_related & prefetch_related 적용 (2/2)

- 문제 해결 2단계

- "102 queries including 100 similar and 100 duplicates" => "2 queries"



선투입 최적화는 악의 근원

"작은 효율성에 대해서는,
말하자면 97% 정도에 대해서는, 잊어버려라.
선투입 최적화는 모든 악의 근원이다."

- 도널드 커누스(Donald E. Knuth)

- ✓ 튜링상을 수상한 컴퓨터 과학자/수학자
- ✓ The Art of Computer Programming의 저자

참고

'exists' method

.exists() 특징

- QuerySet에 결과가 하나 이상 존재하는지 여부를 확인하는 메서드
 - 결과가 포함되어 있으면 True를 반환하고 결과가 포함되어 있지 않으면 False를 반환
 - 데이터베이스에 최소한의 쿼리만 실행하여 효율적
 - 전체 QuerySet을 평가하지 않고 결과의 존재 여부만 확인
 - 해당 데이터를 불러오지 않고, 데이터베이스 수준에서 빠르게 판단 가능
- 대량의 QuerySet에 있는 특정 객체 검색에 유용

exists 적용 예시 (1/2)

- 좋아요 기능에 적용

```
# articles/views.py
@login_required
def likes(request, article_pk):
    article = Article.objects.get(pk=article_pk)
    # if request.user in article.like_users.all():
    if article.like_users.filter(pk=request.user.pk).exists():
        article.like_users.remove(request.user)
    else:
        article.like_users.add(request.user)

    return redirect('articles:index')
```

exists 적용 예시 (2/2)

- 팔로우 기능에 적용

```
# accounts/views.py
@login_required
def follow(request, user_pk):
    User = get_user_model()
    person = User.objects.get(pk=user_pk)
    if person != request.user:
        # if request.user in person.followers.all():
        if person.followers.filter(pk=request.user.pk).exists():
            person.followers.remove(request.user)
        else:
            person.followers.add(request.user)
    return redirect('accounts:profile', person.username)
```


한꺼번에 dump 하기

한꺼번에 dump 하기

```
# 모델 3개를 json 파일 1개로 추출
$ python manage.py dumpdata --indent 4 articles.article articles.comment accounts.user > data.json

# 모든 모델을 json 파일 1개로 추출
$ python manage.py dumpdata --indent 4 > data.json
```

- 다만 모든 데이터를 한 번에 추출 할 경우 파일 용량이 커질 수 있으므로, 필요에 따라 특정 앱만 추출하거나, 파일을 압축하여 관리하는 방법을 고려
- 데이터베이스 변경이 잦은 경우 전체 추출보다는 앱 단위 또는 모델 단위로 관리하는 편이 유지 보수에 용이

loaddata 인코딩 에러

인코딩 문제

- JSON 파일 생성 및 로딩 시, 파일이 특정 문자 인코딩(예: UTF-8)으로 저장되지 않으면 한글 등 비ASCII 문자가 깨지거나, UnicodeDecodeError 등의 에러가 발생할 수 있음
- 윈도우 환경에서 생성한 파일을 리눅스 환경에서 로딩 할 때, 혹은 반대 상황에서 인코딩 이슈가 빈번히 발생

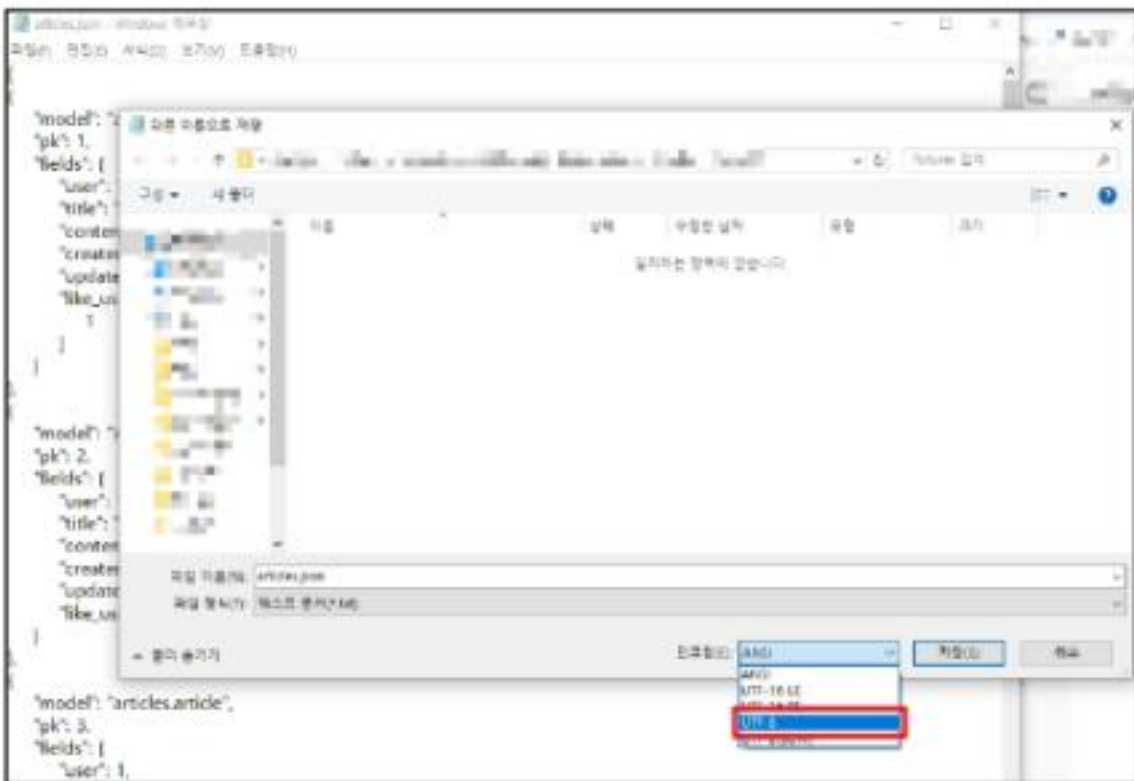
```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc0 in position ...
```


인코딩 문제 해결 방법

- dumpdata 시 추가 명령어 작성

```
$ python -Xutf8 manage.py dumpdata [생략]
```

- 이미 추출된 fixtures 파일이 있다면, 에디터(메모장, VSCode 등)에서 파일을 열고 인코딩을 UTF-8로 지정 한 뒤 다시 저장



<그림20_DB_Many To Many Relationships 02_메모장을 이용한 인코딩 문제 해결>



<그림21_DB_Many To Many Relationships 02_vscode에서 인코딩 문제 해결 방법>

팔로우 기능 구현 (ManyToMany relationship)

- 2925. Auth M:N 복습 - 모델 작성
- 2926. Auth M:N 복습 - 팔로우 기능 구현
- 2607. 소설 연재 서비스 - 모델 정의
- 2384. 소설 연재 서비스 - 구독 서비스
- 2385. 소설 연재 서비스 - 작가 생성 기능 개선
- 3271. 구독 기능 모델 구성하기
- 3272. 구독 기능 구현하기

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 유저가 다른 유저를 팔로우할 때 사용하는 필드는?

- a) OneToOneField
- b) ForeignKey
- c) ManyToManyField
- d) CharField

2 다음 지문을 읽고 정답을 작성하시오.

유저가 유저를 팔로우하는 관계를 설정할 때, 자기 자신을 참조할 때 사용하는 키워드는?

3 다음 문장이 올바른지 O, X로 답하시오.

`symmetrical=True`로 설정하면 유저 A가 유저 B를 팔로우하면, 유저 B도 자동으로 유저 A를 팔로우한다.

4 `related_name`의 주된 역할은?

- a) 필드 이름을 자동 생성
- b) 모델 이름을 바꿔줌
- c) 역참조 시 사용할 이름 지정
- d) 데이터 삭제를 방지

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 다음 지문을 읽고 정답을 작성하시오.

ManyToManyField를 사용하면 자동으로 생성되는 테이블은 무엇을 위한 테이블은?

7 팔로우 기능에서 상대 유저의 정보를 조회할 때 사용하는 함수는?

- a) get_user_model
- b) get_object_or_404
- c) get_user_info
- d) get_profile_data

6 다음 문장이 올바른지 O, X로 답하시오.

ManyToManyField를 설정하면, 별도의 모델을 만들지 않아도 중개 테이블이 생성된다.

8 다음 지문을 읽고 정답을 작성하시오.

유저가 이미 팔로우한 상태인지 확인할 때 사용할 수 있는 QuerySet 메서드는 무엇인가?

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 문장이 올바른지 O, X로 답하십시오.

related_name을 지정하지 않으면 Django는 기본적으로 모델명_set 형식으로 역참조 이름을 만든다.

11 다음 지문을 읽고 정답을 작성하십시오.

ORM에서 반복 접근 시 과도한 쿼리를 발생시키는 문제를 무엇이라 하는가?

10 이미 팔로우한 유저를 다시 팔로우하지 않기 위해 사용하는 조건은?

- a) if request.user == person
- b) if person in request.user.followings
- c) if request.user in person.followers.all()
- d) if person.followers.exists(request.user)

12 다음 문장이 올바른지 O, X로 답하십시오.

annotate는 쿼리셋에 집계 값을 계산해서 새로운 필드를 추가할 수 있다.

Q 각 문제에 올바른 답안을 생각해봅시다

13 select_related는 어떤 관계에서 쿼리 최적화를 위해 사용되나요?

- a) ManyToManyField
- b) CharField
- c) ForeignKey
- d) DateField

14 다음 지문을 읽고 정답을 작성하시오.

ManyToManyField 관계에서 쿼리 최적화를 위해 사용하는 ORM 메서드는 무엇인가?

15 다음 문장이 올바른지 O, X로 답하시오.

loaddata 명령어는 JSON 파일을 DB에 불러오는 데 사용된다.

16 dumpdata 명령어의 주된 기능은?

- a) 데이터 삭제
- b) 데이터 백업
- c) 앱 생성
- d) 마이그레이션 실행

확인 문제



정답 및 해설

1. c) ManyToManyField / 2. 'self' / 3. O / 4. c) 역참조 시 사용할 이름 지정 / 5. 중개 테이블
6. O / 7. a) get_user_model / 8. exists()

1. 한 명의 유저가 여러 명을 팔로우할 수 있고, 여러 명에게 팔로우될 수도 있으므로 ManyToManyField를 사용합니다.
2. ManyToManyField에서 자기 자신과의 관계를 만들기 위해 'self' 키워드를 사용합니다.
3. symmetrical=True는 양방향 관계로, 한쪽에서 연결하면 반대쪽도 자동으로 연결됩니다. 단방향 팔로우는 False로 설정해야 합니다.
4. related_name은 ManyToManyField 등에서 역참조 시 사용할 이름을 지정하여 충돌을 방지하고 가독성을 높여줍니다.

5. ManyToMany 관계에서는 두 모델 간의 연결 정보를 저장하기 위해 Django가 자동으로 중개 테이블을 생성합니다.
6. Django는 ManyToManyField를 설정하면 자동으로 중개 테이블을 생성하므로, 기본적으로 별도 모델을 정의할 필요는 없습니다.
7. Django에서는 사용자 모델을 직접 import하지 않고 get_user_model() 함수를 사용하여 유저 모델 클래스를 가져옵니다.
8. exists() 메서드는 QuerySet 결과가 존재하는지만 확인하므로, 효율적으로 팔로우 여부를 판단할 수 있습니다.

확인 문제



정답 및 해설

9. O / 10. c) if request.user in person.followers.all() / 11. N+1 문제 / 12. O / 13. c) ForeignKey /
14. prefetch_related / 15. O / 16. b) 데이터 백업

9. related_name을 지정하지 않으면 Django는 기본적으로 '모델명소문자_set' 형식으로 역참조 이름을 생성합니다.
예: user_set
10. 이미 팔로우한 유저인지 확인하려면
person.followers.all()에 request.user가 포함되어 있는지 확인합니다.
11. N+1 문제는 기본 데이터 조회 후, 연결된 객체에 접근할 때 N개의 추가 쿼리가 발생하여 성능 저하를 유발하는 현상입니다.
12. annotate는 Count, Sum 등의 집계 함수를 이용해 각 객체에 계산된 값을 담은 필드를 추가합니다.

13. select_related는 ForeignKey 또는 OneToOne 관계에서 JOIN을 사용해 연관 객체를 한 번에 조회하도록 최적화합니다.
14. prefetch_related는 ManyToManyField나 역참조 관계에서 관련 데이터를 미리 가져와 반복 쿼리를 줄이는 데 사용됩니다.
15. loaddata는 dumpdata로 추출한 JSON 파일을 데이터베이스에 로드할 때 사용하는 명령어입니다.
16. dumpdata는 데이터베이스의 데이터를 JSON 등의 형식으로 추출하여 파일로 저장하는 백업 도구입니다.

활동 정리

핵심 키워드

개념	설명	예시
self 관계	모델이 자기 자신을 참조하는 관계	유저 ↔ 유저 (팔로우)
symmetrical	관계의 대칭 여부를 설정하는 속성	친구 관계: True / 팔로우 관계: False
fixtures	JSON이나 YAML 파일로 데이터를 백업하거나 복원하는 Django 기능	python manage.py dumpdata
annotate	집계 함수 결과를 QuerySet에 주석처럼 추가	댓글 개수, 좋아요 수 표시
select_related	ForeignKey 관계의 객체를 미리 로드하여 성능 향상	select_related('author')
prefetch_related	ManyToMany 또는 reverse 관계를 미리 로딩	prefetch_related('comments')
exists	QuerySet에 해당 객체가 존재하는지를 Boolean으로 반환	Comment.objects.filter(...).exists()

<표1_DB_Many To Many Relationships 02_핵심 키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- 팔로우 관계 구현하기
 - 유저 ↔ 유저 구조를 self 관계로 정의하고 symmetrical 속성을 False로 설정
 - 프로필에서 팔로우 여부, 게시글, 댓글 정보 등을 효율적으로 조회 가능
- fixtures로 빠른 데이터 입출력 하기
 - dumpdata와 loaddata를 통해 백업과 복원 작업을 수행할 수 있음
 - 테스트 데이터나 초기 설정 데이터 관리가 쉬워짐
- 쿼리 최적화의 핵심
 - select_related는 FK 관계를, prefetch_related는 N:M/역참조 관계를 최적화함
 - annotate로 집계값을 추가하고, exists로 빠르게 존재 여부 확인 가능

활동 정리

Django에서 유저 관계를 정의하고 쿼리 성능을 높이는 방법을 고민해보자.

- 프로필 페이지에 팔로우 정보와 작성한 콘텐츠를 함께 출력해 사용자 경험을 높일 수 있습니다.

1. 유저 간 팔로우 기능을 구현하려면 자기 자신을 참조하는 관계(self 관계)를 이해해야 합니다.
2. 다대다 관계 구성 시 역참조명 충돌을 방지하기 위해 `related_name` 을 설정합니다.
3. `fixtures(dumpdata/loaddata)`를 활용하면 팔로우 테스트 데이터를 손쉽게 재사용할 수 있습니다.
4. N+1 문제를 해결하려면 `annotate`, `select_related`, `prefetch_related`의 사용 방식에 익숙해져야 합니다.
5. `exists` 메서드를 사용하면 특정 조건에 맞는 데이터가 존재하는지를 효율적으로 판단할 수 있습니다.

- 이러한 기술들은 팔로우 기능뿐만 아니라 유저 기반 상호작용 기능에도 확장 적용할 수 있습니다.
- 학습한 내용을 바탕으로 실무 프로젝트에서 관계 정의와 성능 개선을 동시에 달성할 수 있습니다.

감사합니다

